1 Background

1.1 Score-based generative models

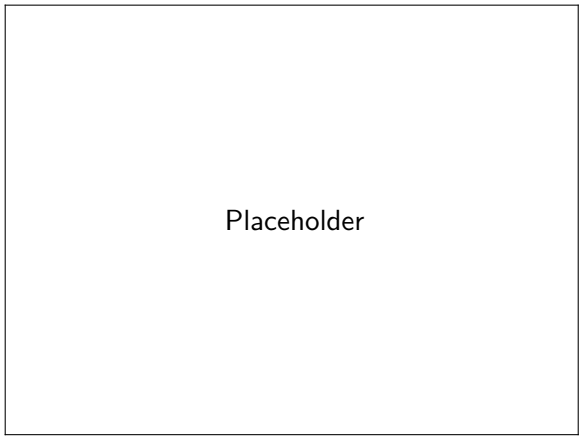


Figure 1: Figure illustrating the forward and reverse diffusion process

- ρ_1: target distribution
- ρ_0: base distribution (typically $\mathcal{N}(0, I)$)
- ρ_t: perturbed data distribution, for $t = 0$ to $t = 1$
- v_t: velocity field, for $t = 0$ to $t = 1$
- $\nabla \log \rho_t(z)$: score function

$$v_t^*(z) = \frac{1}{t}\big(z + (1-t)\nabla \log \rho_t^*(z)\big) \tag{1}$$

- ρ_t^*: marginal distribution of the random variable z, mixture of Gaussians with means tx_i and common covariance matrix $(1-t)^2 I$

$$z = tx + (1-t)\epsilon\text{: marginal distribution of the random variable } z \tag{2}$$

$$x \sim \rho_1 \tag{3}$$

$$(1-t)\epsilon \sim \mathcal{N}(0, (1-t)^2 I) \tag{4}$$

1. Closed-form score:

$$\nabla \log \rho_t^*(z) = \frac{1}{(1-t)^2}(k_t(z) - z), \text{typically learned via score-matching or denoising} \tag{5}$$

$$k_t(z) = \sum_{i=1}^{N} \text{softmax}\left(-\frac{||z - tX||^2}{2(1-t)^2}\right)_i tx_i \tag{6}$$

2. Smoothed closed-form score:

$$s_{\sigma,t}(z) = \frac{1}{(1-t)}\left(\frac{1}{M}\sum_{m=1}^{M} k_t(z+\sigma\epsilon_m) - z\right) \tag{7}$$

- $\sigma \geq 0$: smoothing parameter
- M: noise samples

1.2 Sampling algorithm

$$v_t^*(z) = \frac{1}{t}\big(z + (1-t)s_{\sigma,t}(z)\big) \tag{8}$$

Algorithm 1: Forward Euler scheme sampling

Input : Training set $\{x_i\}_{i=1}^N$, noise $\{u_{i,m}\}$, step size $h = \frac{1}{S}$, initial sample $z_0 = \mathcal{N}(0, I)$

for $n = 0$ **to** $S - 1$ **do**
  $t_n = \frac{n}{S}$
  $z_{n+1} \leftarrow z_n + h v_{\sigma,t_n}(z_n)$
end
return z_T

1.3 Advantages & drawbacks

Advantages:

- Generative model
- Can model multimodal densities
- Works well with highly multi-dimensional data

Drawbacks:

- The dataset needs to be available at inference time (similar as for non-parametric methods like KNN)
- The closed form tends to memorize existing points in the dataset instead of generalizing like the original neural diffusion policies (can be mitigated by using the smoothed score and several M noise samples)
- The closed form uses a kernel so they share the same issues as kernel methods (*i.e.*, complexity of the algorithm is a function of the number of training instances, instead of the number of input dimensions)

- In the experiments they only show state-based policies (*i.e.*, inputs are end-effector positions and object poses), not visuomotor policies (with high-dimensional camera frames)
- The performance is comparable but a bit less good than the original neural diffusion policies